

编译器设计文档

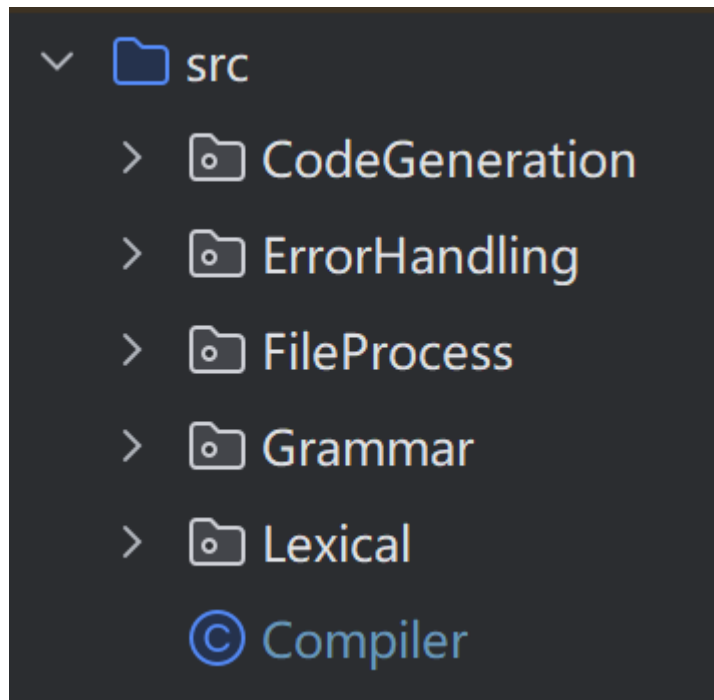
编译器总体设计

所设计的编译器可以从 `testfile.txt` 中读取需要被编译的源程序，在经过词法分析，语法分析，错误处理和代码生成阶段后分别输出到不同的目标文件中。

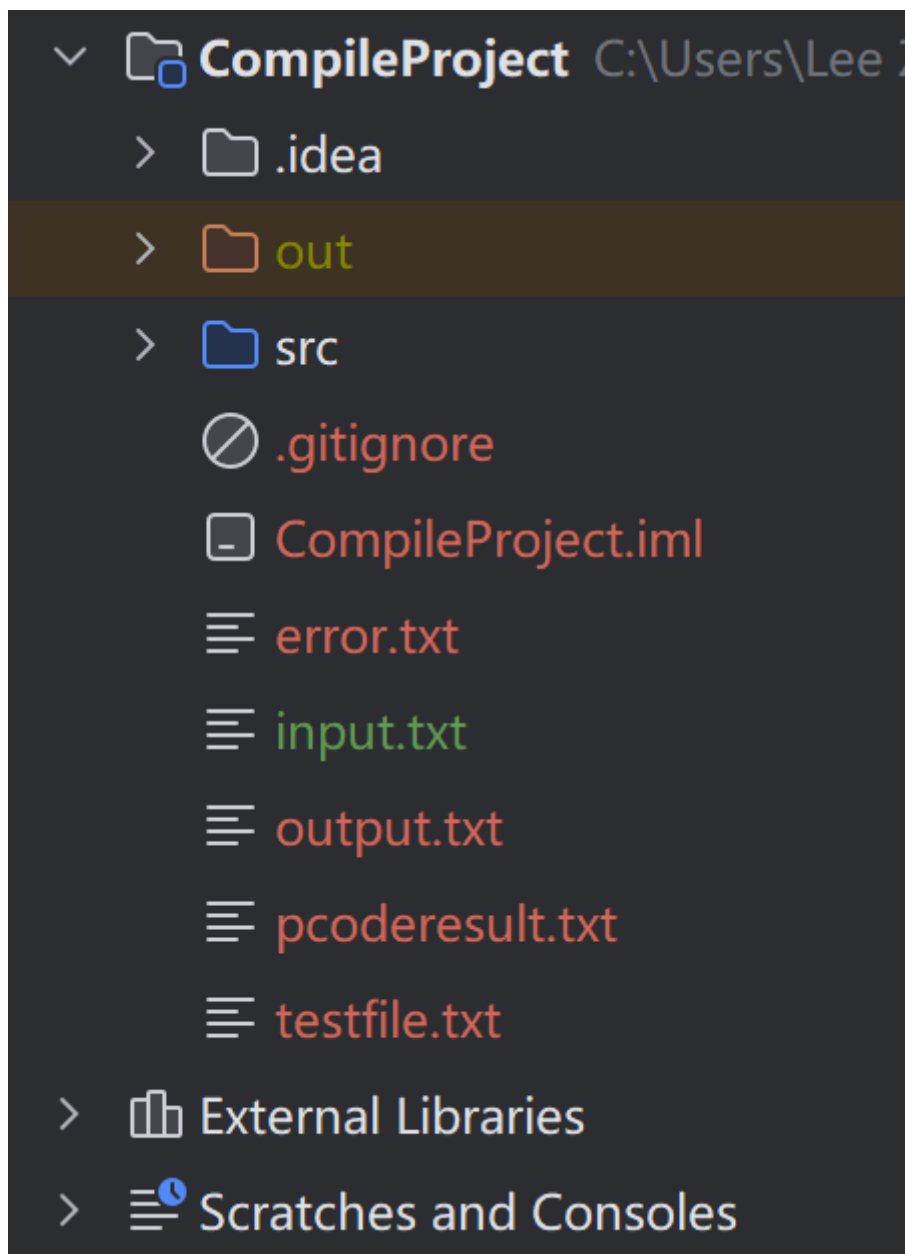
整体架构思路为

- 程序通过 `FileProcess` 包中的 `FileProcessor` 进行外部文件的读取和写入
- `Lexical` 包中的 `Lexer` 主导词法分析阶段
- `Grammar` 包中的 `Parser` 主导语法分析，错误处理和代码生成阶段
- `ErrorHandling` 包中作为存放源程序错误的容器
- `CodeGeneration` 包中的 `PcodeContainer` 作为存放源程序经编译后生成的 `Pcode` 指令的容器；而 `PcodeExecutorVM` 为 `Pcode` 的解释程序
- 以 `Compiler` 中的 `main` 作为程序运行的入口

故 `src` 的结构如下：



等待被编译的源文件 `testfile.txt` 和后续输出文件如：词法分析和语法分析阶段的输出文件 `output.txt`、错误处理的生成文件 `error.txt` 和代码生成和解释阶段的输入和生成文件 `input.txt`，`pcoderesult.txt` 都放在整体 `CompileProject` 根目录下



词法分析设计

编码前设计

目的:

将被编译源程序解析并输出每个单词的类别

架构设计:

被编译源代码经过一个词法分析器可以返回源代码中每一个单词在单词表上所对应的类别码

类别码:

单词名称	类别码	单词名称	类别码	单词名称	类别码	单词名称	类别码
Ident	IDENFR	!	NOT	*	MULT	=	ASSIGN
IntConst	INTCON	&&	AND	/	DIV	;	SEMICN
FormatString	STRCON		OR	%	MOD	,	COMMA

单词名称	类别码	单词名称	类别码	单词名称	类别码	单词名称	类别码
main	MAINTK	for	FORTK	<	LSS	(	LPARENT
const	CONSTTK	getint	GETINTTK	<=	LEQ	)	RPARENT
int	INTTK	printf	PRINTFK	>	GRE	[	LBRACK
break	BREAKTK	return	RETURNTK	>=	GEQ	]	RBRACK
continue	CONTINUETK	+	PLUS	==	EQL	{	LBACE
if	IFTK	-	MINU	!=	NEQ	}	RBRACE
else	ELSETK	void	VOIDTK				

编码后设计

`Lexer` 作为词法分析器，逐个逐行读入被编译源程序的每一个字符构成字符串（单词），从单词表 `LexType` 中找到单词所对应的类别码，将其设为 `tokenNode` 记录在 `list` 中

模块

`LexType` 作为单词表，记录了该编译器的文法中所有单词的类别码（即理论课上所述的终结符）

```
public enum LexType {
    IDENFR,      //Ident
    INTCON,      //IntConst
    STRCON,      //FormatString
    MAINTK,      //main
    CONSTTK,     //const
    INTTK,       //int
    BREAKTK,     //break
    CONTINUETK,  //continue
    IFTK,        //if
    ELSETK,      //else
    NOT,         //!
    AND,         //&&
    OR,          //||
    FORTK,       //for
    GETINTTK,    //getint
    PRINTFK,     //printf
    RETURNTK,    //return
    PLUS,        //+
    MINU,        //-
    VOIDTK,      //void
    MULT,        //*
    DIV,         ///
    MOD,         //%
    LSS,         //<
    LEQ,         //<=
    GRE,         //>
    GEQ,         //>=
    EQL,         //==
    NEQ,         //!=
    ASSIGN,      //=
}
```

```

    SEMICN,    //;
    COMMA,     //,
    LPARENT,   //(
    RPARENT,   //)
    LBRAK,     //[
    RBRAK,     //]
    LBRACE,    //{
    RBRACE,    //{
}

```

单词被识别对应的类别码后，以 `tokenNode` 的形式统一存储

```

public class tokenNode {
    private String content; //记录源程序的单词
    private LexType type;   //记录单词对应的类别码
    private int lineNum;    //记录该单词位于源程序的行数
    private Integer num;    //若单词为数字，记录单词的值

    /***/
}

```

`Lexer` 通过 `getChar` 方法读入字符 `append` 在 `StringBuilder` 单词容器中，在 `generateLexical` 方法中边读边识别 `StringBuilder` 中的内容

`ungetChar` 方法为超前扫描的回退，可以在读入字符时将回退至上一个读入的字符，可用于判断注释符号 `/**` 和 `/*` 和 `*/` 将被注释的内容无视，只留下有效内容。

在 `generateLexical` 中若识别到 `Ident` 类型的单词，先分析其是否为 `reservedwords` 中保留的关键字，若不是被保留的关键字则该 `Ident` 的类别码才是 `IDENFR`

流程：`source` -> `getChar` 读入字符存在 `token` -> 根据 `token` 的内容识别类别码 -> 放入 `list` 容器

```

public class Lexer {
    private final String source; //源程序容器
    private int curPos; //当前位置
    private StringBuilder token; //单词容器
    private int lineNum; //当前行数
    private ArrayList<tokenNode> list; //存储已识别过类别码的单词的容器
    private final HashMap<String, LexType> reservewords = new HashMap<String, LexType>() {
        put("main", LexType.MAINTK);
        put("const", LexType.CONSTTK);
        put("int", LexType.INTTK);
        put("break", LexType.BREAKTK);
        put("continue", LexType.CONTINUETK);
        put("if", LexType.IFTK);
        put("else", LexType.ELSETK);
        put("for", LexType.FORTK);
        put("getint", LexType.GETINTTK);
        put("printf", LexType.PRINTFTK);
        put("return", LexType.RETURNTK);
        put("void", LexType.VOIDTK);
    };
}

```

```

        private final HashMap<String, LexType> operations = new HashMap<String,
LexType>() {{
            put("+", LexType.PLUS);
            put("-", LexType.MINUS);
            put("*", LexType.MULT);
            put("%", LexType.MOD);
        }};

        private final HashMap<String, LexType> brackets = new HashMap<String,
LexType>() {{
            put("{", LexType.LBRACE);
            put("}", LexType.RBRACE);
            put("[", LexType.LBRACK);
            put("]", LexType.RBRACK);
            put("(", LexType.LPARENT);
            put(")", LexType.RPARENT);
        }};

        public Lexer(String source) {
            this.source = source;
            this.curPos = 0;
            this.token = new StringBuilder();
            this.lineNum = 1;
            this.list = new ArrayList<>();
            generateLexical(); //词法分析的入口
        }

        public Character getChar() { //在源程序中读入单个字符
            if (curPos < source.length()) {
                char c = source.charAt(curPos++);
                if (c == '\n') {
                    lineNum++;
                }
                return c;
            }
            return null;
        }

        public void ungetChar() { //超前扫描回退
            curPos--;
            if (source.charAt(curPos) == '\n') {
                lineNum--;
            }
        }

        public void generateLexical() {
            Character c;
            while (curPos < source.length()) {
                clearToken();
                c = getChar();
                if (c == ' ' || c == '\n' || c == '\r' || c == '\t') {
                    continue;
                } else if (Character.isDigit(c)) { //处理数字IntConst

```

```

        do {
            token.append(c);
            c = getChar();
        } while (Character.isDigit(c));
        ungetChar();
        list.add(new tokenNode(token.toString(), LexType.INTCON,
lineNum));
    } else if (Character.isLetter(c) || c == '_') { //处理Ident
        do {
            token.append(c);
            c = getChar();
        } while (Character.isDigit(c) || Character.isLetter(c) || c ==
'_');
        ungetChar();
        if (isReserveWord(token.toString())) { //若为关键字则将Ident归类为对
应关键字的类别码
            list.add(new tokenNode(token.toString(),
getReservedWordLexType(token.toString()), lineNum));
        } else { //若不为关键字则为IDENFR
            list.add(new tokenNode(token.toString(), LexType.IDENFR,
lineNum));
        }
    } else if (isOperation(c.toString())) { //处理算数运算符
        list.add(new tokenNode(c.toString(),
getOperationsLexType(c.toString()), lineNum));
    } else if (isBracket(c.toString())) { //处理括号
        list.add(new tokenNode(c.toString(),
getBracketLexType(c.toString()), lineNum));
    } else if (c == ';') {
        list.add(new tokenNode(c.toString(), LexType.SEMICN, lineNum));
    } else if (c == ',') {
        list.add(new tokenNode(c.toString(), LexType.COMMA, lineNum));
    } else if (isRelation(c)) { //处理关系运算符
        processRelation(c);
    } else if (c == '/') {
        processSlash(c);
    } else if (c == '\\') {
        processFormatString(c);
    }
}

}

private void processRelation(char c) {
    //处理关系运算符
}

private void processSlash(char c) {
    token.append(c);
    Character next = getChar();
    if (next == '/') {
        //若下一个字符也为/, 则该行之后的所有字符都为注释掉的内容, 无视
    } else if (next == '*') {
        //若下一个字符为*, 则不断读入字符并无视掉, 直到遇到下一个*/, 才继续执行
generateLexical
    } else {
        //若不是上述的两种情况, 则该 / 为普通的除符号, 赋予类别码

```

```

        ungetChar();
        list.add(new tokenNode(token.toString(), LexType.DIV, lineNum));
    }
}

private void processFormatString(char c) {
    //处理FormatString -> "....."
    list.add(new tokenNode(token.toString(), LexType.STRCON, lineStart));
}

public void clearToken() { //清空StringBuilder
    token.delete(0, token.length());
}

public LexType getReservedWordLexType(String word) {
    //在reservedwords的map中找到word对应的类别码
}

public LexType getOperationsLexType(String word) {
    //在operations的map中找到word对应的类别码
}

public LexType getBracketLexType(String word) {
    //在brackets的map中找到word对应的类别码
}

public ArrayList<tokenNode> getList() {
    return this.list;
}
}

```

Lexer 词法分析工作完成后，将 list 返回给 FileProcessor 由其将结果打印到 output.txt

```

public class FileProcessor {
    private final FileReader fileReader;
    private final FileWriter parserFileWriter;
    private final String fileName = "testfile.txt";
    private final String outputName = "output.txt";
    private final String source;

    public FileProcessor() throws IOException {
        this.fileReader = new FileReader(fileName);
        this.parserFileWriter = new FileWriter(outputName);
        source = generateSource();
    }

    private String generateSource() {
        StringBuilder sb = new StringBuilder();
        String line;

        try(Scanner scanner = new Scanner(fileReader)) {
            while (scanner.hasNextLine()) {
                line = scanner.nextLine();
                sb.append(line).append("\n");
            }
        }
    }
}

```

```

    }
    return sb.toString();
}

public String getSource() {
    return source;
}

public FileWriter getParserFileWriter() {
    return parserFileWriter;
}

public void generateLexerOutput(ArrayList<tokenNode> nodes) throws
IOException {
    for (tokenNode node : nodes) {
        parserFileWriter.append(node.getLexType().toString());
        parserFileWriter.append(" ");
        parserFileWriter.append(node.getContent());
        //filewriter.append(" ");
        //filewriter.append(String.valueOf(node.getLineNum()));
        //filewriter.append(" ");
        //filewriter.append(String.valueOf(node.getNum()));
        parserFileWriter.append("\n");
    }
    parserFileWriter.flush();
    parserFileWriter.close();
}
}

```

Compiler中

```

public class Compiler {
    public static void main(String[] args) throws IOException {
        FileProcessor fp = new FileProcessor();
        Lexer lexer = new Lexer(fp.getSource());    //通过FileProcessor将source给
        Lexer
        fp.generateLexerOutput(lexer.getList());    //再由Lexer将解析后的list传回给
        FileProcessor进行输出
    }
}

```

语法分析设计

目的:

词法分析已将源码中所有单词的类别码识别出，也就意味着文法中的终止符已在源程序中标上，语法分析将对类别码根据文法作推导，在推导的过程中边建立源程序的AST语法树

文法:

```

编译单元 CompUnit → {Decl} {FuncDef} MainFuncDef
声明 Decl → ConstDecl | VarDecl
常量声明 ConstDecl → 'const' BType ConstDef { ',' ConstDef } ';'
基本类型 BType → 'int'

```

```

常数定义 ConstDef → Ident { '[' ConstExp ']' } '=' ConstInitVal
常量初值 ConstInitVal → ConstExp | '{' [ ConstInitVal { ',' ConstInitVal } ] '}'
变量声明 VarDecl → BType VarDef { ',' VarDef } ';'
变量定义 VarDef → Ident { '[' ConstExp ']' } | Ident { '[' ConstExp ']' } '='
InitVal
变量初值 InitVal → Exp | '{' [ InitVal { ',' InitVal } ] '}'
函数定义 FuncDef → FuncType Ident '(' [FuncFParams] ')' Block
主函数定义 MainFuncDef → 'int' 'main' '(' ')' Block
函数类型 FuncType → 'void' | 'int'
函数形参表 FuncFParams → FuncFParam { ',' FuncFParam }
函数形参 FuncFParam → BType Ident '[' ']' { '[' ConstExp ']' }
语句块 Block → '{' { BlockItem } '}'
语句块项 BlockItem → Decl | Stmt
语句 Stmt → LVal '=' Exp ';'
        | [Exp] ';'
        | Block
        | 'if' '(' Cond ')' Stmt [ 'else' Stmt ]
        | 'for' '(' [ForStmt] ';' [Cond] ';' [ForStmt] ')' Stmt
        | 'break' ';' | 'continue' ';'
        | 'return' [Exp] ';'
        | LVal '=' 'getint' '(' ')' ';'
        | 'printf' '(' FormatString{',' Exp} ')' ';'
语句 ForStmt → LVal '=' Exp
表达式 Exp → AddExp
条件表达式 Cond → LOrExp
左值表达式 LVal → Ident { '[' Exp ']' }
基本表达式 PrimaryExp → '(' Exp ')' | LVal | Number
数值 Number → IntConst
一元表达式 UnaryExp → PrimaryExp | Ident '(' [FuncRParams] ')' | UnaryOp UnaryExp
单目运算符 UnaryOp → '+' | '-' | '!'
函数实参表 FuncRParams → Exp { ',' Exp }
乘除模表达式 MulExp → UnaryExp | MulExp ('*' | '/' | '%') UnaryExp
加减表达式 AddExp → MulExp | AddExp ('+' | '-') MulExp
关系表达式 RelExp → AddExp | RelExp ('<' | '>' | '<=' | '>=') AddExp
相等性表达式 EqExp → RelExp | EqExp ('==' | '!=') RelExp
逻辑与表达式 LAndExp → EqExp | LAndExp '&&' EqExp
逻辑或表达式 LOrExp → LAndExp | LOrExp '||' LAndExp
常量表达式 ConstExp → AddExp

```

编码前设计

采用自顶向下的递归下降分析法来构造语法树，而实验规定的文法中含有左递归死循环，需要将其进行消除后才可以进行分析

左递归文法如下：

```

加减表达式 AddExp → MulExp | AddExp ('+' | '-') MulExp
乘除模表达式 MulExp → UnaryExp | MulExp ('*' | '/' | '%') UnaryExp
逻辑或表达式 LOrExp → LAndExp | LOrExp '||' LAndExp
逻辑与表达式 LAndExp → EqExp | LAndExp '&&' EqExp
相等性表达式 EqExp → RelExp | EqExp ('==' | '!=') RelExp
关系表达式 RelExp → AddExp | RelExp ('<' | '>' | '<=' | '>=') AddExp

```

将文法改写成BNF范式：

```

加减表达式 AddExp → MulExp { ('+' | '-') MulExp }
乘除模表达式 MulExp → UnaryExp { ('*' | '/' | '%') UnaryExp }
逻辑或表达式 LORExp → LAndExp { '||' LAndExp }
逻辑与表达式 LAndExp → EqExp { '&&' EqExp }
相等性表达式 EqExp → RelExp { ('==' | '!=') RelExp }
关系表达式 RelExp → AddExp { ('<' | '>' | '<=' | '>=') AddExp }

```

编码后设计

Grammar 中的 Parser 作为语法分析器，按照文法对源程序作语法分析，设立一个 interface 接口 ASTNode 作为语法树上的节点，语法树的节点会分为终止符和非终止符，因此让 NonTerminalNode 类和在词法分析时所生成记录终止符的 tokenNode 实现 ASTNode 接口，并让所有的非终止符类继承 NonTerminalNode 父类

```

public interface ASTNode {
    void print(Filewriter filewriter) throws IOException;
}

```

NonTerminalNode 类

```

public class NonTerminalNode implements ASTNode {
    private final ArrayList<ASTNode> childNodes;

    public NonTerminalNode() {
        this.childNodes = new ArrayList<>();
    }

    public void addChild(ASTNode node) {
        childNodes.add(node);
    }

    public ArrayList<ASTNode> getChildNodes() {
        return this.childNodes;
    }

    @Override
    public void print(Filewriter filewriter) throws IOException {

    }
}

```

tokenNode

```

public class tokenNode implements ASTNode {
    private String content;
    private LexType type;
    private int lineNum;
    private Integer num;

    /*...*/

    @Override
    public void print(Filewriter filewriter) throws IOException {

```

```

        filewriter.append(type.toString());
        filewriter.append(" ");
        filewriter.append(content);
        filewriter.append("\n");
    }
}

```

非终止符类 (以 `CompUnit` 为例)

```

public class CompUnit extends NonTerminalNode {
    public CompUnit() {
        super();
    }

    public void addChild(ASTNode node) {
        super.addChild(node);
    }

    public void print(Filewriter filewriter) throws IOException {
        Iterator<ASTNode> iterator = super.getChildNodes().iterator();
        while (iterator.hasNext()) {
            iterator.next().print(filewriter);
        }
        filewriter.append("<CompUnit>\n");
    }
}

```

Parser 语法分析器中 `list` 用于接收从 `Lexer` 词法分析器所生成的 `tokenNode`，通过 `curToken` 以 `next` 方法将 `tokenNode` 依序提取，`root` 作为整棵 `AST` 语法树的根节点，自顶向下以递归下降的方法构建语法树 (以 `CompUnit` 举例)，终结符为该 `AST` 语法树的叶节点，因此在文法的推导过程中遇到终结符时通过 `setLeafNode` 方法将其设置为叶节点

```

public class Parser {
    private final ArrayList<tokenNode> list;
    private tokenNode curToken;
    private int pos;
    private CompUnit root;

    public Parser(ArrayList<tokenNode> list) {
        this.list = list;
        this.pos = 0;
        this.curToken = list.get(pos);
        root = parseCompUnit(); //递归下降的入口
    }

    public void next() {
        if (pos < list.size() - 1) {
            pos++;
            curToken = list.get(pos);
        }
    }

    public CompUnit parseCompUnit() {
        //CompUnit -> {Decl} {FuncDef} MainFuncDef
    }
}

```

```

    CompUnit compUnit = new CompUnit();

    //不是函数 Decl
    while (list.get(pos + 1).getType().compareTo(LexType.MAINTK) != 0
           && list.get(pos + 2).getType().compareTo(LexType.LPARENT) != 0) {
        compUnit.addChild(parseDecl());
    }

    //是函数 FuncDef
    while (list.get(pos + 1).getType().compareTo(LexType.MAINTK) != 0
           && list.get(pos + 2).getType().compareTo(LexType.LPARENT) == 0) {
        compUnit.addChild(parseFuncDef());
    }

    //MainFuncDef
    if (curToken.getType().compareTo(LexType.INTTK) == 0
        && list.get(pos + 1).getType().compareTo(LexType.MAINTK) == 0) {
        compUnit.addChild(parseMainFuncDef());
        return compUnit;
    } else {
        error();
    }
    return null;
}

/*文法其他非终结符的分析*/

public tokenNode setLeafNode() {
    tokenNode leaf = curToken;
    if (pos < list.size() - 1) {
        pos++;
        curToken = list.get(pos);
    }
    return leaf;    //返回叶节点
}
}

```

语法分析结束后将 root 交给 FileProcessor 由其进行输出

```

public class FileProcessor {
    //....
    private final FileWriter parserFileWriter;
    private final String fileName = "testfile.txt";
    private final String outputName = "output.txt";

    public FileProcessor() throws IOException {
        this.fileReader = new FileReader(fileName);
        this.parserFileWriter = new FileWriter(outputName);
        this.errorsFileWriter = new FileWriter(errorName);
        this.pcodeResultFileWriter = new FileWriter(resultName);
        source = generateSource();
    }

    public void generateParserOutput(CompUnit root) throws IOException {
        root.print(parserFileWriter);
    }
}

```

```
        parserFileWriter.flush();
        parserFileWriter.close();
    }
}
```

Compiler 中

```
public class Compiler {
    public static void main(String[] args) throws IOException {
        FileProcessor fp = new FileProcessor();
        //...
        Parser parser = new Parser(lexer.getList());
        fp.generateParserOutput(parser.getRoot());
        //...
    }
}
```

错误处理设计

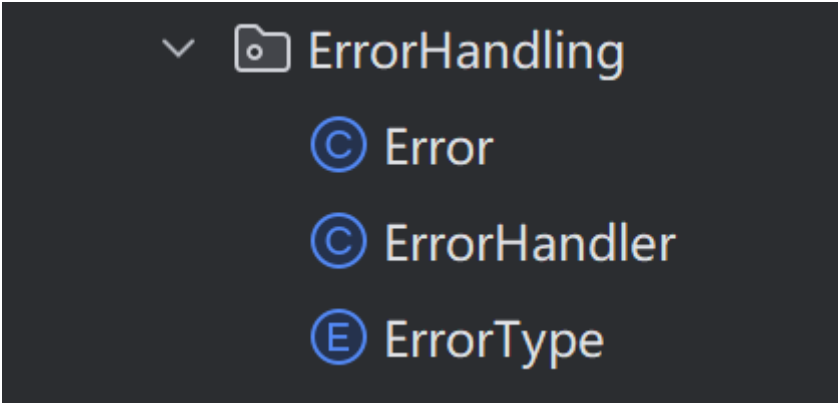
编码前设计

错误处理功能需要将源程序中的语法错误或语义错误返回给用户，实验中需要实现的可能的错误有：

错误类型	错误类别码	解释
非法符号	a	格式字符串中出现非法字符报错行号为 所在行数。
名字重定义	b	函数名或者变量名在 当前作用域 下重复定义。注意，变量一定是同一级作用域下才会判定出错，不同级作用域下，内层会覆盖外层定义。报错行号为 所在行数。
未定义的名字	c	使用了未定义的标识符报错行号为 所在行数。
函数参数个数不匹配	d	函数调用语句中，参数个数与函数定义中的参数个数不匹配。报错行号为函数调用语句的 函数名 所在行数。
函数参数类型不匹配	e	函数调用语句中，参数类型与函数定义中对应位置的参数类型不匹配。报错行号为函数调用语句的 函数名 所在行数。
无返回值的函数存在不匹配的return语句	f	报错行号为 'return' 所在行号。
有返回值的函数缺少return语句	g	只需要考虑函数末尾是否存在return语句， 无需考虑数据流 。报错行号为函数 结尾的' 所在行号。
不能改变常量的值	h	为常量时，不能对其修改。报错行号为 所在行号。
缺少分号	i	报错行号为分号 前一个非终结符 所在行号。
缺少右小括号')'	j	报错行号为右小括号 前一个非终结符 所在行号。

错误类型	错误类别码	解释
缺少右中括号']'	k	报错行号为右中括号前一个非终结符所在行号。
printf中格式字符与表达式个数不匹配	l	报错行号为 'printf' 所在行号。
在非循环块中使用 break和continue语句	m	报错行号为 'break' 与 'continue' 所在行号。

该部分有两种实现的方法，一种是直接在 parser 语法解析器中直接在递归下降子程序时中处理错误部分，另一种则是重新遍历整棵语法树，单独进行错误处理。在面向对象的编程中，显然后者的方法更适合，可以让代码更为模块化，处理事情的先后更有序，且更符合面向对象的主旨想法，将模块的复杂度尽可能降低。但我在此时并没有想到有这样的方法，一头直接就莽地选择了最容易想到，也就是第一种的方法，所幸在词法和语法阶段的架构设计的较好，没有给我在这个部分因复杂度的增加的调试增加太大的难度。



本阶段新增了一个 `ErrorHandling` 包，包内的 `Error` 类用来记录源程序中含有的错误，由于实验要求输出附带行数，因此增加一个行数的属性用于记录。 `ErrorType` 作为记录了所有错误类别码的 `Enum` 错误表，源程序中出现的所有错误由 `ErrorHandler` 作为一个容器存储，遍历结束后 `ErrorHandler` 将结果返回，并由 `FileProcessor` 输出到 `error.txt`

`Error`

```
public class Error {
    private final ErrorType errorType; //记录错误类型
    private final int lineNumber; //记录错误发生行数

    public Error(ErrorType errorType, int lineNumber) {
        this.errorType = errorType;
        this.lineNum = lineNumber;
    }

    public ErrorType getErrorType() {
        return errorType;
    }

    public int getLineNum() {
        return lineNum;
    }
}
```

```
}
```

ErrorType

```
public enum ErrorType {  
    a,  
    b,  
    c,  
    d,  
    e,  
    f,  
    g,  
    h,  
    i,  
    j,  
    k,  
    l,  
    m  
}
```

ErrorHandler

```
public class ErrorHandler {  
    private static final ArrayList<Error> errorList = new ArrayList<>();  
  
    public static void addError(Error error) {  
        for (Error e : errorList) {  
            if (e.getLineNum() == error.getLineNum()) {  
                return;  
            }  
        }  
        if (!errorList.isEmpty() &&  
            errorList.get(errorList.size() - 1).getLineNum() >  
            error.getLineNum()) {  
            for (int i = 0; i < errorList.size(); i++) {  
                if (error.getLineNum() < errorList.get(i).getLineNum()) {  
                    errorList.add(i, error);  
                    break;  
                }  
            }  
        } else {  
            errorList.add(error);  
        }  
    }  
  
    public static ArrayList<Error> getList() {  
        return errorList;  
    }  
  
    public static void print() {  
        for (Error e : errorList) {  
            System.out.println(e.getErrorType() + " : " + e.getLineNum());  
        }  
    }  
}
```

```
}
```

编码后设计

错误处理中需要生成符号表来检查针对函数和常量变量的错误（如重定义，未定义，赋值给常量等），在 Grammar 包中新增 Table 包存放符号表。

SymbolManager 用以处理变量和常量，FunctionManager 用以记录函数。为保证符号表管理者的单一性，对 SymbolManager 和 FunctionManager 采用单例模式。

每进入一个新的块需要开设新的一个符号表，并在该块结束后将表弹出，SymbolManager 中的 symbolTables 是一个存储符号表 ArrayList<Symbol> 的 ArrayList 容器，每个 Symbol 表示一个符号（变量或常量），ArrayList<Symbol> 表示该作用域的符号表，在初始化时，预先将一个 ArrayList<Symbol> 加入 symbolTables 作为全局常量和变量的符号表，之后在每次需要开设新的 Block 时，就加入一个新的符号表并在 Block 解析结束后将该 Block 的符号表弹出栈，简而言之，时刻保持栈顶为当前块的符号表。

SymbolManager

```
public class SymbolsManager {
    private static SymbolsManager instance = new SymbolsManager();
    private final ArrayList<ArrayList<Symbol>> symbolTables = new ArrayList<>()
    {{
        add(new ArrayList<>());
    }}

    private SymbolsManager() {}

    public static SymbolsManager getInstance() {
        return instance;
    }

    public void addBlock() {    //加入新的块的符号表
        symbolTables.add(new ArrayList<>());
    }

    public void quitBlock() {    //将结束分析的块的符号表弹出
        symbolTables.remove(symbolTables.size() - 1);
    }

    /*
        ....
    */
}
```

Symbol

```

public class Symbol {
    private LexType varType;    //int 或 const
    private String varName;    //变量名称

    /*
        ....
    */
}

```

FunctionManager 用来管理函数的定义，参数的个数和类型等，每当遍历一个除了 main 之外的函数定义时，将该函数的信息登记在 FunctionManager 中的 functionsTable 中，functionsTable 是一个存储 Function 函数信息的 HashMap，以函数名和对应的 Function 作为键值对存储在内存。Function 中的 parameters 记录参数的个数和类型，用 Integer 的

FunctionManager

```

public class FunctionsManager {
    private static FunctionsManager instance = new FunctionsManager();
    private final HashMap<String, Function> functionsTable = new HashMap<>();
    //函数定义表
    private Function currentFunctionBlock = null;    //当前定义函数

    private FunctionsManager() {
    }

    public static FunctionsManager getInstance() {
        return instance;
    }

    public void newFunction() { //开新的函数信息表
        currentFunctionBlock = new Function();
    }
    public void addFunction() { //将函数信息表登记
        functionsTable.put(currentFunctionBlock.getFuncName(),
currentFunctionBlock);
    }

    public Function writeFunction() {    //返回当前函数信息表，对当前函数信息表做编辑
        return currentFunctionBlock;
    }
}

```

Function

```

public class Function {
    private LexType funcType;    //函数类型void 或 int
    private String funcName;    //函数名
    private boolean noReturn = false;    //是否有return
    private final ArrayList<Integer> parameters;    //记录函数的参数个数和类型

    public Function() {
        this.funcType = null;
        this.funcName = null;
        this.parameters = new ArrayList<>();
    }
}

```

```

}

public Function(LexType funcType, String funcName) {
    this.funcType = funcType;
    this.funcName = funcName;
    this.parameters = new ArrayList<>();
}

public LexType getFuncType() {
    return this.funcType;
}

public void setFuncType(LexType funcType) {
    this.funcType = funcType;
    this.noReturn = true;
}

/*
    ....
*/
}

```

有了以上两类表便可以解决 b, c, d, e, f, g, h 类问题；a 类错误可在词法解析器中由其检测错误；i, j, k 类错误可在 Parser 语法分析器中递归下降时处理并报错。

为解决 m 类问题，可以在 Parser 语法分析器中设置一个 inLoop 属性，在解析到 Stmt 并进入循环内时将 inLoop 属性 +1，退出循环时将 inLoop 属性 -1，在解析到 break 或 continue 时可通过判断 inLoop 属性的值判断目前是否位于循环内来决定是否报错。

```

public class Parser {
    //.....
    private int inLoop = 0;

    private void getInToLoop() {
        this.inLoop++;
    }

    private void getOutFromLoop() {
        this.inLoop--;
    }

    public Stmt parseStmt() {
        /*....*/
        if (curToken.getLexType().compareTo(LexType.FORTK) == 0) {
            //'for' '(' [ForStmt] ';' [Cond] ';' [ForStmt] ')' Stmt

            /*
                ...
            */
            /*进入循环*/
            getInToLoop();
            /*
                ...
            */
            /*从循环退出*/

```

```

        getOutFromLoop();
    }

    if (curToken.getLexType().compareTo(LexType.BREAKTK) == 0
        || curToken.getLexType().compareTo(LexType.CONTINUETK) == 0) {
        //'break' ';' | 'continue' ';'
        /*
            ...
        */
        if (inLoop <= 0) {
            ErrorHandler.addError(new Error(ErrorType.m,
curToken.getLineNum()));
        }
        /*
            ...
        */
    }
}
}

```

1 类问题可通过设置一个 `getFormatChar` 方法获取 `strcon` 中有多少个 `%d`，并获取 `strcon` 后的 `Exp` 个数判断是否一致。

```

public class Parser {
    /*
        ...
    */
    private int getFmtChar(String strcon) {
        return strcon中有多少个%d
    }
}

```

错误检测结束后由 `FileProcessor` 将错误输出到 `error.txt`

```

public class FileProcessor {
    private final FileReader fileReader;
    private final FileWriter errorsFileWriter;
    private final String fileName = "testfile.txt";
    private final String errorName = "error.txt";

    public FileProcessor() throws IOException {
        this.fileReader = new FileReader(fileName);
        this.parserFileWriter = new FileWriter(outputName);
        this.errorsFileWriter = new FileWriter(errorName);
        this.pcodeResultFileWriter = new FileWriter(resultName);
        source = generateSource();
    }

    public void generateErrorOutput() throws IOException {
        for (Error error : ErrorHandler.getList()) {
            errorsFileWriter.append(String.valueOf(error.getLineNum()));
            errorsFileWriter.append(" ");
            errorsFileWriter.append(error.getErrorType().toString());
            errorsFileWriter.append("\n");
        }
    }
}

```

```

    }
    errorsFileWriter.flush();
    errorsFileWriter.close();
}
}

```

Compiler 中

```

public class Compiler {
    public static void main(String[] args) throws IOException {
        FileProcessor fp = new FileProcessor();
        //.....
        fp.generateErrorOutput();
        //.....
    }
}

```

代码生成设计

我所选择的目标代码为 Pcode，为此在这个代码生成阶段需要完成 Pcode 的生成和解释程序，在输出文件中 pcodeResult.txt 展示源程序代码的执行结果。

为此我们需要一个 PcodeContainer 用于存储源代码程序所生成的 Pcode，还需要一个可以解释 Pcode 的解释程序，我所选择的是仿照 p10 编译器解释程序的栈式虚拟机，为此我们需要设计一系列的 Pcode 指令

指令	格式	说明
LIT	LIT imm	将 imm 立即数加载到栈顶
OPR	OPR OPRTYPE	进行栈顶和次栈顶（/单独栈顶）的运算，OPRTYPE 决定运算类型，并把结果存在栈顶
LOD	LOD level addr	栈顶作为动态偏移，加载运行栈中 (level, addr) + 动态偏移 处的值到栈顶
STO	STO level addr	栈顶作为动态偏移，将次栈顶的值存储到运行栈中的 (level, addr) + 动态偏移 处
CALL	CALL LABEL	调用 LABEL 函数（需分配AR，设置静态和动态链，传递参数）
BLKS	BLKS	开启新的 Block
BLKE	BLKE	结束当前 Block
JMP	JMP LABEL	跳转到 LABEL 处
JPF	JPF LABEL	若栈顶元素为 0，则跳转到 LABEL 处
JPT	JPT LABEL	若栈顶元素不为 0，则跳转到 LABEL 处
INT	INT imm	栈顶指针增加 imm
RED	RED	从 input.txt 中读入数字到栈顶

指令	格式	说明
WRT	WRT	将栈顶元素输出到 pcodeResult.txt
WRTS	WRTS str	将 str 字符串输出到 pcodeResult.txt
LABEL	LABEL label	标记作用
RET	RET	回收 AR，返回调用该函数的地方
LEA	LEA level addr	将 (level, addr) 处元素的绝对地址加载到栈顶

OPRType

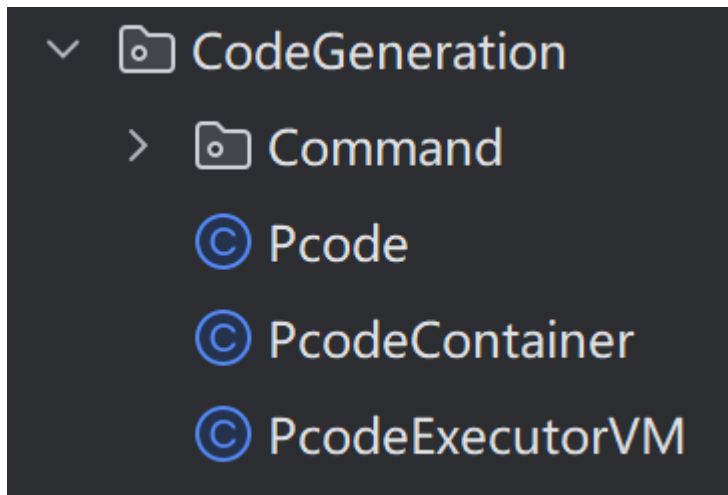
OPRType	运算符
ADD	+
SUB	-
MULT	*
DIV	/
MOD	%
CMPEQ	==
CMPNEQ	!=
CMPGE	>=
CMPLE	<=
CMPGT	>
CMPLT	<
NOT	!

在代码的生成阶段我们通过以上指令去对被编译的源程序代码进行 Pcode 的生成，在 Pcode 的生成阶段我们会需要再一次使用到符号表

代码生成阶段与错误处理一样可以有两种方法去完成这一阶段的任务，一是在递归下降子程序中边生成语法树边生成 Pcode 代码；二是重新遍历已生成好的语法树，同样的前者会增加代码的复杂度，我选择的依然是在递归子程序中生成 Pcode 代码。

编码前设计

引入 CodeGeneration 包：



`Pcode` 是一个父类，`Command` 包中含有所有以上继承了 `Pcode` 的指令类，在 `Pcode` 的生成阶段将生成的代码存储在 `PcodeContainer` 中，`PcodeContainer` 采用单例模式，并且在存储 `Pcode` 时若存储的 `Pcode` 类型为 `LABEL`，将其记录在 `LabelsRecorder` 中，`LabelsRecorder` 是一个以 `LABEL` 所在的 `pc` 值和 `LABEL` 名作为键值对存储的 `HashMap`，以便在代码解释阶段可以直接从中找出要跳转到的 `pc` 值

`Pcode`

```
public class Pcode {
    private String command; //指令种类

    public Pcode(String command) {
        this.command = command;
    }

    public String getCommand() {
        return this.command;
    }

    public void printPcode() {

    }
}
```

`Command` 中继承了 `Pcode` 父类的指令（以 `LOD` 为例）

```
public class LOD extends Pcode {
    private int level;
    private int addr;

    public LOD(int level, int addr) {
        super("LOD");
        this.level = level;
        this.addr = addr;
    }

    @Override
    public void printPcode() {
        System.out.println(super.getCommand() + " " + level + " " + addr);
    }

    public int getLevel() {
```

```

        return level;
    }

    public int getAddr() {
        return addr;
    }
}

```

PcodeContainer

```

package CodeGeneration;

import CodeGeneration.Command.LABEL;

import java.util.ArrayList;
import java.util.HashMap;

public class PcodeContainer {
    private static final PcodeContainer instance = new PcodeContainer();
    private final ArrayList<Pcode> codes;
    private final HashMap<String, Integer> labelsRecorder;

    private PcodeContainer() {
        this.codes = new ArrayList<>();
        this.labelsRecorder = new HashMap<>();
    }

    public static PcodeContainer getInstance() {
        return instance;
    }

    public void addPcode(Pcode pcode) {
        codes.add(pcode);

        if (pcode instanceof LABEL) {
            labelsRecorder.put(((LABEL)pcode).getLabel(), codes.indexOf(pcode));
//            System.out.println(labelsRecorder);
        }
    }

    public ArrayList<Pcode> getCodes() {
        return this.codes;
    }

    public HashMap<String, Integer> getLabelsRecorder() {
        return this.labelsRecorder;
    }

    public void printPcode() {
        int i = 0;
        for (Pcode pcode : codes) {
            System.out.print(i++ + " ");
            pcode.printPcode();
        }
    }
}

```

```
}
```

编码后设计

主要改动的比较大的地方在于 LABEL 指令的记录和使用，考虑到 if-else，for 情况的嵌套，需要记录外层的 LABEL，在这里我所采用的解决方案是在 PcodeContainer 中管理所有 LABEL

对于 if-else：

每个 if-else 的 stmt 一定生成三个 LABEL：if_start_label，else_start_label，if_end_label

其标记的顺序为：

```
Cond //该ifStmt的短路求值判断
if_start_label:

//...(if的stmt)

JMP if_end_label
else_start_label:

//...(else的stmt)

if_end_label:
```

在短路求值中若关系表达式为真则执行 if_start_label 的 stmt，执行完后 JMP 到 if_end_label 表示结束；若关系表达式为假则跳转到 else_start_label，执行到 if_end_label 表示该 if-else 结束

为了区分每一个 LABEL 避免出现同样 LABEL 的情况，PcodeContainer 中含有每个 LABEL 各自的计数器，LABEL 格式为：

```
return "if_Label_" + if_LabelCnt;
```

考虑到嵌套的可能性，每一次生成 LABEL 后先将 LABEL 压入各自的栈，每次仅从栈顶弹出 LABEL 供使用，这样可以保证在嵌套的情况下提取的 LABEL 一直是属于当前 if-else 的 LABEL

```
private int if_LabelCnt = 0;
private int ifEnd_LabelCnt = 0;
private int else_LabelCnt = 0;
private final ArrayList<String> ifLabelStack = new ArrayList<>();
private final ArrayList<String> ifEndLabelStack = new ArrayList<>();
private final ArrayList<String> elseLabelStack = new ArrayList<>();

//以if_Label为例
public void generateIfLabel() {
    ifLabelStack.add(getIfLabelNo());
}

public String getIfLabel() {
    return ifLabelStack.remove(ifLabelStack.size() - 1);
}

public String getIfLabelNo() {
    String ret = "if_Label_" + if_LabelCnt;
```

```

        if_LabelCnt++;
        return ret;
    }

    //ifEnd_Label 和 else_Label 同上

```

对于 for 的情况也一样，每一次解析到 stmt 为 for 类型时都会生成三个 LABEL：forStart_Label，forStmt_Label，forEnd_Label

标记顺序为：

```

//forStmt1 (i = 0/....)

for_start_label:

//Cond

JPF for_end_label

//Stmt...

for_stmt_label:

//forStmt2(i = i + 1/....)

JMP for_start_label
for_end_label:

```

需要对 for 的第二个 forStmt 特地标个标记是因为考虑到 continue 的情况下该 forStmt 需要被执行，因此需要被跳转到该地址

为了区分 LABEL 避免有一样的 LABEL 情况，采用的方法和 if-else 一样

```

private int forStart_LabelCnt = 0;
private int forStmt_LabelCnt = 0;
private int forEnd_LabelCnt = 0;
private final ArrayList<String> forStart_LabelStack = new ArrayList<>();
private final ArrayList<String> forStmt_LabelStack = new ArrayList<>();
private final ArrayList<String> forEnd_LabelStack = new ArrayList<>();

//以 forStart_Label 为例
public void generateForStart_Label() {
    forStart_LabelStack.add(getForStart_LabelNo());
}

public String getForStart_LabelNo() {
    String ret = "forStart_Label_" + forStart_LabelCnt;
    forStart_LabelCnt++;
    return ret;
}

public String getForStmt_Label() {
    return forStmt_LabelStack.get(forStmt_LabelStack.size() - 1);
}

```

```
//forEnd_Label 和 forStmt_Label 同上
```

代码解释设计

代码解释阶段是根据代码生成阶段所生成的代码进行解释，这个阶段基本上和被编译的源程序没什么关系了，都是在和生成的代码打交道，因此也不存在什么语法树，符号表的说法，由此可见该阶段极度依赖代码生成阶段的成果。本人采用的是栈式虚拟机的方式来对生成的 Pcode 进行解释。

运行栈中每个 AR 的前4个地址是被保留的，分别为返回值，返回地址，静态链和动态链，每次分配新的 AR 时需要根据情况设置前四个参数，如函数调用时需要考虑设置返回地址，返回值，静态链和动态链，开启新的块时需要设置静态链。

编码前设计

给虚拟机设置运行栈所需的指针：

PcodeExecutorVM

```
public class PcodeExecutorVM {
    private static PcodeExecutorVM instance = new PcodeExecutorVM();    //单例模式
    private ArrayList<Pcode> pcodes;    //被编译程序生成的Pcode
    private int stack[];    //运行栈
    private int sp; //指向栈顶的指针
    private int pc; //程序计数器
    private int mp; //作用域基地址
    private HashMap<String, Integer> labelsRecorder;    //从PcodeContainer接收的
    label表
    private Scanner scanner;
    private final StringBuilder result = new StringBuilder();    //打印到
    pcoderesult.txt

    private PcodeExecutorVM() {
        this.stack = new int[1000000];    //一开始先设置容量为100000的运行栈
        stack[0] = 0;
        stack[1] = -1;
        stack[2] = 0;
        stack[3] = 0;
        this.sp = 3;    //设置栈顶
        this.mp = 0;    //当前作用域基地址
        this.pc = 0;    //程序计数器设为0
        this.pcodes = PcodeContainer.getInstance().getCodes();    //接收生成的Pcode
        this.scanner = new Scanner(System.in);
        this.labelsRecorder = PcodeContainer.getInstance().getLabelsRecorder();
    //接收label表
    }

    public void startExecute() {
        while(pc < pcodes.size()) {
            run(pcodes.get(pc));
        }
    }

    private void run(Pcode pcode) {
        if (pcode instanceof BLKS) {
```

```

    } else if (pcode instanceof BLKE) {

    } else if (pcode instanceof CALL) {

    } else if (pcode instanceof RET) {

    } else if (pcode instanceof LOD) {

    } else if (pcode instanceof STO) {

    } else if (pcode instanceof JPT) {

    } else if (pcode instanceof JPF) {

    } else if (pcode instanceof JMP) {

    } else if (pcode instanceof LIT) {

    } else if (pcode instanceof INT) {

    } else if (pcode instanceof LEA) {

    } else if (pcode instanceof RED) {

    } else if (pcode instanceof WRT) {

    } else if (pcode instanceof WRTS) {

    } else if (pcode instanceof OPR) {

    } else if (pcode instanceof LABEL) {

    }

}

```

编码后设计

发现到运行栈可能会有容量不足的情况，为运行栈增加一个动态扩容的代码

```

if (sp >= stack.length - 1) {
    int[] arrayNew = new int[stack.length + 10000];
    System.arraycopy(stack, 0, arrayNew, 0, stack.length);
    stack = arrayNew;
}

```

解释程序的虚拟机结束工作后同样由 `FileProcessor` 将结果输出到 `pcoderesult.txt`

`FileProcessor`

```

package FileProcess;

import ErrorHandling.Error;
import ErrorHandling.ErrorHandler;
import Grammar.GrammarNode.NonTerminate.CompUnit;
import Lexical.tokenNode;

```

```

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Scanner;

public class FileProcessor {
    //.....
    private final FileWriter pcodeResultFileWriter;
    private final String resultName = "pcodeResult.txt";

    //.....
    public void generatePcodeResult(String executeResult) throws IOException {
        pcodeResultFileWriter.append(executeResult);
        pcodeResultFileWriter.flush();
        pcodeResultFileWriter.close();
    }
}

```

Compiler

```

public class Compiler {
    public static void main(String[] args) throws IOException {
        FileProcessor fp = new FileProcessor();

        //.....

        PcodeExecutorVM.getInstance().startExecute();
        fp.generatePcodeResult(PcodeExecutorVM.getInstance().getExecuteResult());
    }
}

```

总结感想

本学期的编译技术课程我认为挺好的，实验课每一个阶段的安排我认为难度都适中，是属于那种有挑战性，但不至于束手无策，很无助的程度，即便是到了代码生成这般困难的阶段，课程组所安排的专题报告也对我们打开思路有一定的帮助。有了上个学期 oo 课程的经验，我在写编译器的起初阶段花费了比较多的心思在架构，将编译器的可扩展性做的比较好，以致于在后面的阶段对编译器作迭代开发时比较轻松。我个人感觉这个编译器的开发是非常有成就感的，这应该是我大学以来写过代码量最大的一份作业。理论课上所学到的还是比较抽象，但是有这个实验课把理论课的知识具象化，让我对这些知识点理解的更深刻，并有机会把所学习到的实际应用出来。在最后的收尾阶段，回过头来看自己写的编译器成就感满满，我想这应该会是对我来说意义很深刻的一份作业，是我对这一学期编译课的一份答卷。在这里要感谢课程组的付出和辛苦，在我们学习编译课的路上为我们保驾护航，起到一个很好的辅助角色。助教团也辛苦了，为我们准备了答疑课和专题报告都很有收获。